Bézier Curve — Algebraic Expansion (Symbolic)

Code Explanation for Julia Beginners

`02-bezier-sympy.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the Julia code in `02-bezier-sympy.jl` for readers new to Julia. The script computes the same Bézier curve as `01-bezier-algebra.jl` but replaces the hand-written `Poly` type with **Symbolics.jl** — Julia's computer algebra system. This document is fully self-contained: all concepts are explained from scratch even where they overlap with the first script.

Contents

1 Type Map: How the Pieces Fit Together **2**

2 Imports and the Symbolic Variable **2**
2.1 `using Symbolics` 2
2.2 `@variables u` — creating a symbolic variable 3

3 Exact Arithmetic: `Rational{Int}` **3**
3.1 What is `Rational{Int}`? 3
3.2 The typed array literal `Rational{Int}[...]` 4
3.3 Why exact arithmetic matters here 4

4 The `BezierExpansion` Struct **4**
4.1 Why `Vector` and `Dict{String,Any}`? 4

5 The `BezierExpansion` Constructor **5**
5.1 Building the basis: `expand()` 5
5.2 Building per-axis polynomials 5

6 The `evaluate` Function **6**

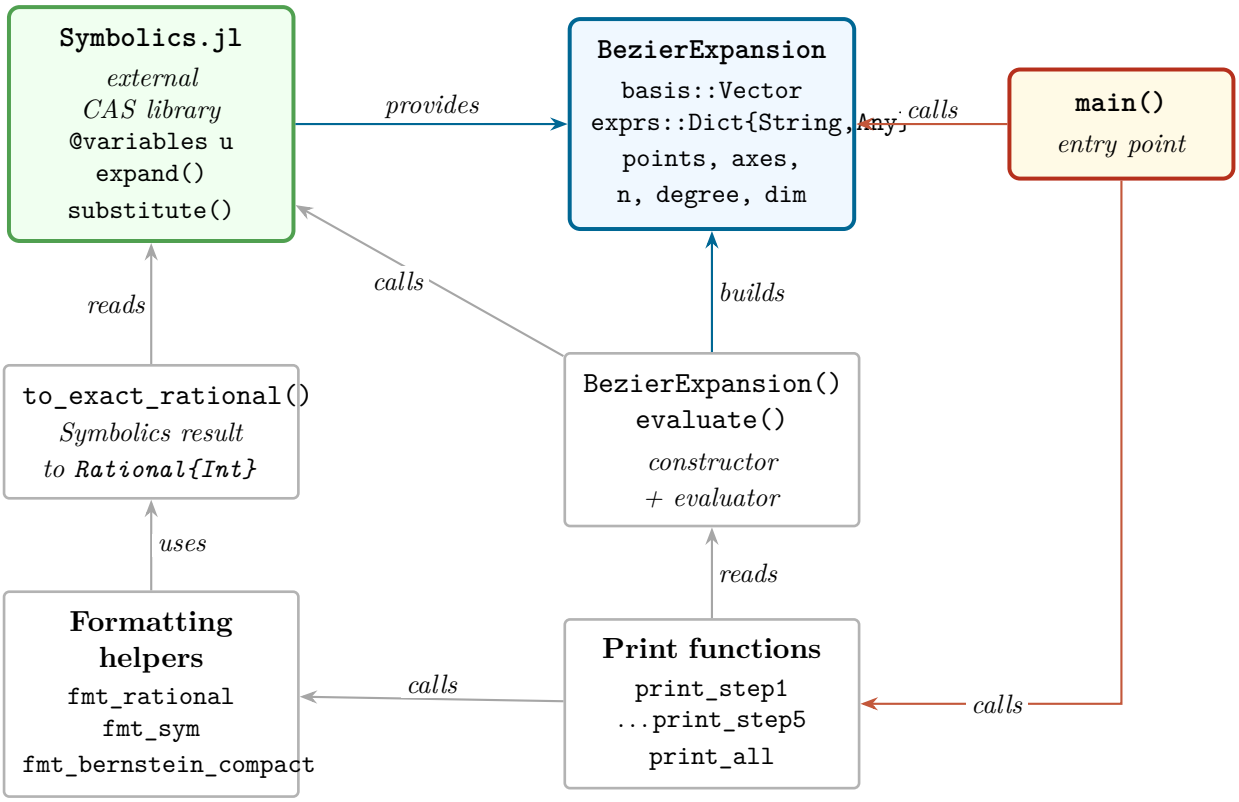
7 Formatting Helpers **6**
7.1 `fmt_rational` — multiple dispatch on two types 6
7.2 `fmt_sym` — formatting a Symbolics expression 7
7.3 `to_exact_rational` — converting a CAS result to a fraction 7
7.4 `fmt_bernstein_compact` 8

8 Print Step Functions **8**
8.1 What is the same 8
8.2 Step 5: the new evaluation loop 9

9 Summary: What Symbolics.jl Replaced **10**

1. Type Map: How the Pieces Fit Together

Script 02 has a simpler structure than Script 01. The hand-rolled `Poly` *type and all its operator overrides are gone. In their place stands a single call to* `expand()` *from Symbolics.jl, which does the same work in one line. The diagram below shows what remains.*



Key difference from Script 01. Script 01 had a `Poly` struct with overridden operators (`*`, `+`, `(p)(u)`) sitting between the constructor and the results. Here, **Symbolics.jl** (green box) replaces all of that. `expand()` does what `expand_bernstein` did. `substitute()` does what `(p)(u)` did. The arrow from `BezierExpansion()` directly to Symbolics.jl shows that the CAS is the new computation engine.

2. Imports and the Symbolic Variable

```
using Symbolics
using Printf

@variables u
```

2.1. `using Symbolics`

`using Symbolics` loads the Symbolics.jl package — Julia's built-in computer algebra system. It provides the ability to work with symbolic mathematical expressions: polynomials, equations, and expressions containing variables like `u` that have no fixed numeric value.

2.2. `@variables u` — creating a symbolic variable

`@variables u` is a **macro** call. In Julia, macros begin with `@` and transform code before it runs. This particular macro, provided by Symbolics.jl, declares `u` as a **symbolic variable**.

After this line, `u` is not a number. It is a placeholder — a name that represents an unknown value. You can build expressions with it:

```
@variables u
expr = 3*u^2 - 6*u + 1
```

`expr` is now the symbolic expression $3u^2 - 6u + 1$. Julia does not compute a number — it holds the expression as a symbolic object.

You can then substitute a value later:

```
substitute(expr, Dict(u => 1//2))
```

This gives the exact rational result $3 \cdot \frac{1}{4} - 6 \cdot \frac{1}{2} + 1 = \frac{3}{4} - 3 + 1 = -\frac{5}{4}$.

Why a macro and not a function?

A regular function call like `declare_variable("u")` would create a variable with that string name, but you could not then write `u` in your code and have Julia know you mean that variable. The macro `@variables u` rewrites the source code so that the Julia identifier `u` is bound to a symbolic object. That is something only a macro can do — it operates on the code itself before the program runs.

3. Exact Arithmetic: `Rational{Int}`

```
const CASE_DATA = Dict(
    2 => (
        points   = Rational{Int}[0 0 1;
                                 0 4 1;
                                 4 0 1;
                                 4 4 1;
                                 5 4 1],
        u_values = Rational{Int}[0, 1//4, 1//2, 3//4, 1],
        axes     = ["x", "y", "z"],
    ),
)
```

3.1. What is `Rational{Int}`?

`Rational{Int}` is Julia's built-in type for exact fractions. It stores a numerator and denominator as plain integers, with no floating-point representation involved.

The `//` operator creates a `Rational{Int}` value:

Expression	Value	Type
`1//4`	$\frac{1}{4}$	`Rational{Int}`
`3//4`	$\frac{3}{4}$	`Rational{Int}`
`4//1`	4	`Rational{Int}` (same as integer 4, but exact)
`1//3`	$\frac{1}{3}$	`Rational{Int}` — no rounding, ever

Compare: `Float64(1/3) = 0.3333333333333333` (rounded). `1//3` is exact.

3.2. The typed array literal `Rational{Int}[...]`

Writing `Rational{Int}[0 0 1; 0 4 1; ...]` tells Julia to create a matrix where every element is stored as `Rational{Int}`, even though the values look like plain integers. So `4` is stored as `4//1`, not as `Int64(4)` or `Float64(4.0)`.

This matters because all arithmetic that follows — multiplying coordinates by basis polynomials, summing terms — will stay in exact rational form throughout. No floating-point error can creep in.

3.3. Why exact arithmetic matters here

The Bernstein basis coefficients are binomial coefficients (always integers). The control point coordinates are whole numbers stored as `Rational{Int}`. The u values are simple fractions: $0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$.

All of these are rational numbers. Their polynomial combinations are therefore also rational numbers. Script 02 *computes* and *displays* those exact fractions — for example $\frac{269}{256}$ — rather than approximating them as `1.050781`.

> **Compared to Script 01:** Script 01 stored coordinates as `Float64` and all arithmetic was floating-point. Script 02 stores coordinates as `Rational{Int}` and all arithmetic is exact integer fraction arithmetic. The decimal output at the end is the same, but the path to get there is fundamentally different.

4. The `BezierExpansion` Struct

```julia
struct BezierExpansion
    points  ::Matrix{Rational{Int}}
    axes    ::Vector{String}
    n       ::Int
    degree  ::Int
    dim     ::Int
    basis   ::Vector        # symbolic expressions per B(n,i,u)
```

```
    exprs    ::Dict{String,Any}  # collected symbolic expr per axis
end
```

Compare this to Script 01's struct:

What changed in the struct:

Field	Script 01	Script 02
`points`	`Matrix{Float64}`	`Matrix{Rational{Int}}`
`basis`	`Vector{Poly}`	`Vector` (untyped — holds Symbolics expressions)
`polys`	`Dict{String,Poly}`	renamed to `exprs::Dict{String,Any}`

The fields `axes`, `n`, `degree`, `dim` are identical.

4.1. Why `Vector` and `Dict{String,Any}`?

In Script 01, `basis` was `Vector{Poly}` — a vector where every element is a `Poly` object, a type we defined ourselves.

In Script 02, `basis` is just `Vector` with no type parameter, and `exprs` values are `Any`. This is because Symbolics.jl expressions have complex internal types that are awkward to name explicitly. Saying `Any` tells Julia: "I don't want to restrict the type here, accept whatever Symbolics.jl returns."

This is less strict than Script 01, but it is the practical approach when working with a CAS whose expression types are not simple to predict.

5. The `BezierExpansion` Constructor

```
function BezierExpansion(points::Matrix{Rational{Int}},
                         axes::Vector{String})
    n   = size(points, 1)
    deg = n - 1
    dim = length(axes)

    basis = [
        expand(binomial(deg, i) * u^i * (1 - u)^(deg - i))
        for i in 0:n-1
    ]

    exprs = Dict{String,Any}()
    for (dim_idx, axis) in enumerate(axes)
        expr = sum(basis[i+1] * points[i+1, dim_idx] for i in 0:n-1)
        exprs[axis] = expand(expr)
    end

    BezierExpansion(points, axes, n, deg, dim, basis, exprs)
end
```

This constructor does in about ten lines what Script 01 did with `expand_bernstein`, `Poly +`, `Poly *`, and the `BezierExpansion` constructor combined. The CAS absorbs all of that complexity.

5.1. Building the basis: `expand()`

```
basis = [
    expand(binomial(deg, i) * u^i * (1 - u)^(deg - i))
    for i in 0:n-1
]
```

This is a **list comprehension** — it evaluates the expression in brackets for each value of `i` from `0` to `n-1` and collects the results into a vector.

For each `i`, it builds the Bernstein polynomial $B(n, i, u) = \binom{n}{i} u^i (1-u)^{n-i}$ symbolically, then calls `expand()` to distribute and collect all terms.

For $n = 4$ (quartic), $i = 2$:

```
expand(binomial(4, 2) * u^2 * (1 - u)^2)
```

`binomial(4, 2) = 6` (Julia built-in, exact integer)

The expression before expanding: $6 \cdot u^2 \cdot (1-u)^2$

After `expand()`: $6u^2 - 12u^3 + 6u^4$

Symbolics.jl returns this as a symbolic expression object. In Script 01, the same result came from `expand_bernstein(4, 2)`, which did the binomial expansion manually step by step. Here, one call to `expand()` handles everything.

5.2. Building per-axis polynomials

```
exprs = Dict{String,Any}()
for (dim_idx, axis) in enumerate(axes)
    expr = sum(basis[i+1] * points[i+1, dim_idx] for i in 0:n-1)
    exprs[axis] = expand(expr)
end
```

For each axis, it computes the weighted sum of all basis polynomials, each multiplied by the corresponding coordinate.

`sum(...for i in 0:n-1)` is Julia's generator sum syntax: it accumulates $\sum_{i=0}^{n-1} B_i \cdot \text{coord}_i$ without building an intermediate array.

The final `expand(expr)` collects all like powers of `u` into a clean monomial polynomial.

For the x axis, Case 2:

x-coordinates are $0, 0, 4, 4, 5$.

$$\text{expr} = B_0 \cdot 0 + B_1 \cdot 0 + B_2 \cdot 4 + B_3 \cdot 4 + B_4 \cdot 5$$

After `expand()`:

$$x(u) = 24u^2 - 32u^3 + 13u^4$$

Stored as `exprs["x"]` — a Symbolics expression, not a `Poly` object.

6. The `evaluate` Function

```
function evaluate(exp::BezierExpansion, u_val::Rational{Int})
    [substitute(exp.exprs[ax], Dict(u => u_val)) for ax in exp.axes]
end
```

`substitute(expr, Dict(u => u_val))` is the Symbolics.jl way of evaluating a symbolic expression at a specific value. It replaces every occurrence of the symbolic variable `u` with the given rational value `u_val`.

The `Dict(u => u_val)` syntax creates a substitution map: "replace `u` with `u_val`."

Evaluating $x(u)$ at $u = \frac{1}{2}$:

```
substitute(exprs["x"], Dict(u => 1//2))
```

This replaces u with $\frac{1}{2}$ in $24u^2 - 32u^3 + 13u^4$:

$$24 \cdot \tfrac{1}{4} - 32 \cdot \tfrac{1}{8} + 13 \cdot \tfrac{1}{16} = 6 - 4 + \tfrac{13}{16} = \tfrac{45}{16}$$

The result is a Symbolics expression representing $\frac{45}{16}$. It is not yet a plain Julia `Rational{Int}` — that conversion happens in `to_exact_rational()`.

Compared to Script 01: Script 01 evaluated a polynomial by calling `(p)(u_val)`, which used Horner's method on the `Float64` coefficient vector. Script 02 calls `substitute()`, which is symbolic substitution — the CAS replaces the variable and simplifies algebraically. The result is an exact rational, not a float.

7. Formatting Helpers

7.1. `fmt_rational` — multiple dispatch on two types

```
fmt_rational(r::Rational{Int}) =
    isone(denominator(r)) ? string(numerator(r)) : string(r)
fmt_rational(n::Integer) = string(n)
```

This is a compact example of **multiple dispatch**: two methods with the same name but different argument types. Julia picks the right one based on what you pass in.

First method handles a `Rational{Int}`:

- `denominator(r)` returns the bottom of the fraction.
- `isone(denominator(r))` checks if it equals 1.

- If the denominator is 1, the fraction is a whole number — print just the numerator (e.g. `"4"` not `"4//1"`).
- Otherwise print the full fraction (e.g. `"269//256"`).

Second method handles any plain `Integer` (like `Int64`): just convert to string directly.

```
fmt_rational(4//1) → "4"    (denominator is 1, so whole number)
fmt_rational(269//256) → "269//256"    (denominator is 256)
fmt_rational(5) → "5"    (plain integer, second method)
```

7.2. `fmt_sym` — formatting a Symbolics expression

```
fmt_sym(expr) = string(expr)
```

This one-liner converts any Symbolics.jl expression to a human-readable string. It has no type annotation on `expr` — it accepts anything and calls Julia's built-in `string()` on it, which Symbolics.jl has extended to produce readable output.

This is how `(24//1)*(u^2) - (32//1)*(u^3) + (13//1)*(u^4)` ends up in the printed output.

7.3. `to_exact_rational` — converting a CAS result to a fraction

```
function to_exact_rational(val)
    v = Symbolics.value(val)
    try
        r = rationalize(Int, Float64(v); tol=1e-12)
        return r
    catch
        return v
    end
end
```

After `substitute()` returns a Symbolics expression like `45//16`, this function extracts the plain Julia `Rational{Int}` value from it. This is needed because Symbolics.jl wraps even simple numeric results in its own expression type, and we want a plain fraction we can format and convert to `Float64`.

The steps are:

1. `Symbolics.value(val)` — strips the Symbolics wrapper, returning a lower-level numeric type.
2. `Float64(v)` — converts to a float temporarily.
3. `rationalize(Int, ...; tol=1e-12)` — converts the float back to the nearest `Rational{Int}` within a tolerance of 10^{-12}. This recovers the exact fraction, since the float was exact to begin with (it came from rational arithmetic).
4. The `try/catch` handles any edge cases where the conversion fails — in that case the raw value `v` is returned as-is.

Tracing `to_exact_rational` for $x\left(\frac{1}{2}\right)$:

After `substitute`, `val` is a Symbolics expression representing $\frac{45}{16}$.

`Symbolics.value(val)` → a SymbolicUtils numeric for 2.8125

`Float64(v)` → `2.8125`

`rationalize(Int, 2.8125; tol=1e-12)` → `45//16`

Result: `45//16` as a plain Julia `Rational{Int}`.

Why go through Float64 and back?

It seems roundabout to convert a rational to float and then back to rational. The reason is that `Symbolics.value()` does not always return a type with a direct `Rational` conversion path — going through `Float64` and `rationalize` is the reliable general approach. For the values in this program (small integer fractions), the round-trip is exact.

7.4. fmt_bernstein_compact

```
const SUPERSCRIPTS = Dict(0=>"", 1=>"", 2=>"^2", 3=>"^3",
                          4=>"^4", 5=>"^5")

function fmt_bernstein_compact(n::Int, i::Int)
    b      = binomial(n, i)
    m      = n - i
    u_part = i == 0 ? "" : (i == 1 ? "u" :
    ↪  "u$(get(SUPERSCRIPTS,i,"^$i"))")
    v_part = m == 0 ? "" : (m == 1 ? "(1-u)" :
                            "(1-u)$(get(SUPERSCRIPTS,m,"^$m"))")
    b_str  = b == 1 ? "" : string(b)
    result = "$(b_str)$(u_part)$(v_part)"
    isempty(result) ? "1" : result
end
```

This produces compact human-readable symbolic strings like `"6u^2(1-u)^2"` for the print steps. It is structurally identical to `fmt_bernstein_symbolic` in Script 01, with one extra guard at the end: `isempty(result) ? "1" : result`. This handles the edge case $B(0, 0, u) = 1$, which would otherwise produce an empty string.

Call	Formula	Output
`fmt_bernstein_compact(4, 0)`	$(1-u)^4$	`"(1-u)^4"`
`fmt_bernstein_compact(4, 1)`	$4u(1-u)^3$	`"4u(1-u)^3"`
`fmt_bernstein_compact(4, 2)`	$6u^2(1-u)^2$	`"6u^2(1-u)^2"`
`fmt_bernstein_compact(4, 4)`	u^4	`"u^4"`

8. Print Step Functions

The five print step functions in Script 02 do exactly the same job as in Script 01 — they display the five stages of the algebraic expansion. The structure of each function is identical. What changed is which fields they read from `exp` *and which formatting helpers they call.*

8.1. What is the same

All five functions follow the same pattern: call `print_step_header`, loop over axes and/or control points, call a formatting helper, call `println`. None of them return a value.

Step	Script 01 source	Script 02 source
Step 1	`fmt_bernstein_symbolic`	`fmt_bernstein_compact`
Step 2	`fmt_bernstein_symbolic` + `fmt_coeff`	`fmt_bernstein_compact` + `fmt_rational`
Step 3	`fmt_poly(exp.basis[i+1])`	`fmt_sym(exp.basis[i+1])`
Step 4	`expand(basis[i]*coord)` via `Poly*`	`expand(exp.basis[i]*coord)` via Symbolics
Step 5	`evaluate` returns `Float64`	`evaluate` returns Symbolics expr, then `to_exact_rational`

8.2. Step 5: the new evaluation loop

The evaluation loop in Step 5 is the most changed part:

```
for u_val in u_values
    results = evaluate(exp, u_val)
    row = "  " * lpad(fmt_rational(u_val), col_u)
    for val in results
        r     = to_exact_rational(val)
        exact = fmt_rational(r)
        dec   = @sprintf("%.6f", Float64(r))
        row  *= "  " * lpad(exact, col_exact) *
                "  " * lpad(dec, col_dec)
    end
    println(row)
end
```

For each `u_val`, it calls `evaluate` (which calls `substitute` internally), then for each result:

1. `to_exact_rational(val)` — extract the exact fraction
2. `fmt_rational(r)` — format as `"45//16"` or `"5"`
3. `Float64(r)` — convert to decimal for the second column

This produces the two-column output (exact + decimal) that distinguishes Script 02's evaluation table from Script 01's.

One row traced: $u = \frac{1}{4}$

`fmt_rational(1//4)` → `"1//4"` (printed in the u column)

For the x axis:

1. `evaluate` calls `substitute(exprs["x"], Dict(u => 1//4))` → Symbolics result for $\frac{269}{256}$
2. `to_exact_rational` extracts `269//256`
3. `fmt_rational(269//256)` → `"269//256"`
4. `Float64(269//256)` → `1.05078125` → `"1.050781"`

Row printed: `1//4  269//256  1.050781  ...`

9. Summary: What Symbolics.jl Replaced

Script 01 (hand-written)	Script 02 (Symbolics.jl)
`struct Poly` with `coeffs::Vector{Float64}`	Symbolics expression (no separate type)
`Poly * scalar` (operator override)	`expr * coord` (Symbolics handles `*`)
`Poly + Poly` (operator override, zero-padding)	`expr + expr` (Symbolics handles `+`)
`expand_bernstein(n, i)` (manual binomial steps)	`expand(binomial(n,i) * u^i * (1-u)^m)`
`(p)(u)` functor via Horner's method	`substitute(expr, Dict(u => val))`
`Float64` results	`Rational{Int}` exact fractions
One column per axis in evaluation table	Two columns: exact fraction + decimal

Which is better?

Neither — they make different trade-offs.

Script 01 is self-contained: it needs no external library, every operation is explicit and transparent, and it is easy to follow step by step.

Script 02 is more powerful: it produces exact results, handles any rational input without rounding, and the constructor is dramatically shorter because the CAS absorbs all the polynomial algebra.

> *Script 01 teaches you how the engine works. Script 02 lets you use the engine without building it. Both are worth understanding.*

Code explanation for `02-bezier-sympy.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated